

Project Assignment 2

Worcester Polytechnic Institute

Tracy Yang

Kyle Yee

July 13, 2026

1 Fix the Joints

1.1 Overview

For this assignment, joints 1 and 2 were fixed so that only the last joint (joint 3, the prismatic joint) remains active. This was done by changing the `type` attribute of `joint_1` and `joint_2` from `revolute` to `fixed` in the URDF.

1.2 URDF Frame Correction

The PA1 feedback noted that the URDF and FK frames did not match. The root cause was that `joint_1`'s origin was placed at the bottom of the column ($z=0.05$), while the DH convention places Frame 1 at the shoulder (top of the column). This split the d_1 offset across two joints (`joint_1` and the fixed `arm_1_attach`), causing a mismatch.

The fix was to move `joint_1`'s origin to $z=0.5$ ($= 0.05_{\text{base}} + 0.45_{\text{column}}$), placing Frame 1 directly at the shoulder where a_1 begins. The `link_1` visual origin was adjusted to $z=-0.225$ so the column geometry remains in the correct world position. The DH parameters now match the URDF exactly:

Table 1: SCARA DH Parameters (corrected)

Parameter	Symbol	Value
Joint 1 z -offset (base to shoulder)	d_1	0.50 m
Arm 1 length	a_1	0.40 m
Link 2 length	a_2	0.30 m
Joint 3 range	d_3	$[-0.2, 0.2]$ m

Listing 1: Fixed URDF joints. Joint 1 origin corrected to $z=0.5$; joints 1 and 2 set to fixed type.

```
1 <!-- joint_1: FIXED, origin raised to z=0.5 to match DH Frame 1 -->
```

```

2 <joint name="joint_1" type="fixed">
3   <parent link="base_link"/><child link="link_1"/>
4   <origin xyz="0 0 0.5"/>
5   <axis xyz="0 0 1"/>
6 </joint>
7
8 <!-- arm_1_attach: origin collapsed to 0 (offset now in joint_1) -->
9 <joint name="arm_1_attach" type="fixed">
10   <parent link="link_1"/><child link="arm_1"/>
11   <origin xyz="0 0 0"/>
12 </joint>
13
14 <!-- joint_2: FIXED -->
15 <joint name="joint_2" type="fixed">
16   <parent link="arm_1"/><child link="link_2"/>
17   <origin xyz="0.4 0 0"/>
18   <axis xyz="0 0 1"/>
19 </joint>
20
21 <!-- joint_3: ACTIVE (prismatic, last joint) -->
22 <joint name="joint_3" type="prismatic">
23   <parent link="link_2"/><child link="link_3"/>
24   <origin xyz="0.3 0 0"/>
25   <axis xyz="0 0 1"/>
26   <limit effort="1000" lower="-0.2" upper="0.2" velocity="0.5"/>
27 </joint>

```

2 Position Controller Node

2.1 Architecture Overview

The controller was implemented as a new ROS2 Python package (`scara_controller`). The overall architecture is:

- **Subscriber:** `/joint_states` (`sensor_msgs/JointState`) — reads `joint_3` position from Gazebo via the `ros_gz_bridge`.
- **Publisher:** `/scara/joint3_cmd` (`std_msgs/Float64`) — publishes the computed PD effort to Gazebo via bridge.
- **Service:** `/scara/set_joint3_position` (`scara_ik/SetJointPosition`) — accepts a reference position and stores it as the controller setpoint.
- **Control loop:** 100 Hz timer executing the PD law.

The effort value is delivered to Gazebo through the `gz-sim-joint-position-controller-system` plugin with gains set to $p = 1$, $i = 0$, $d = 0$, making it act as a direct pass-through. This ensures the force computed by the ROS2 node is applied exactly as-is inside the Gazebo

physics loop, which resolves the WSL2 clock desynchronization that prevented reliable force application via `apply_joint_effort`.

2.2 Reading Joint States (Part 2a)

The node subscribes to `/joint_states`, which is bridged from the Gazebo joint state publisher plugin. The `joint_3` position is extracted by name lookup:

Listing 2: Joint state subscriber callback

```
1 def joint_state_cb(self, msg: JointState):
2     """Read joint_3 position from the Gazebo-bridged joint state."""
3     if 'joint_3' in msg.name:
4         idx = msg.name.index('joint_3')
5         self.cur_position = msg.position[idx]
```

The Gazebo joint state publisher plugin (in `scara.urdf`) publishes only `joint_3` since joints 1 and 2 are fixed:

Listing 3: Gazebo joint state publisher plugin (URDF)

```
1 <gazebo>
2   <plugin filename="gz-sim-joint-state-publisher-system"
3       name="gz::sim::systems::JointStatePublisher">
4     <topic>/world/empty/model/scara/joint_state</topic>
5     <joint_name>joint_3</joint_name>
6   </plugin>
7
8   <plugin filename="gz-sim-joint-position-controller-system"
9       name="gz::sim::systems::JointPositionController">
10    <joint_name>joint_3</joint_name>
11    <topic>/scara/joint3_cmd</topic>
12    <p_gain>1</p_gain>
13    <i_gain>0</i_gain>
14    <d_gain>0</d_gain>
15  </plugin>
16 </gazebo>
```

2.3 PD Controller Design (Part 2b)

A PD controller was designed for `joint_3`. Since no analytical model was required, the gains were tuned empirically. The control law is:

$$u(t) = K_p e(t) + K_d \dot{e}(t)$$

where $e(t) = q_{\text{ref}} - q_{\text{cur}}$ is the position error and $\dot{e}(t)$ is approximated by a finite difference at each timestep:

$$\dot{e}(t) \approx \frac{e(t) - e(t - \Delta t)}{\Delta t}$$

The tuned gains are $K_p = 800$ N/m and $K_d = 80$ N·s/m.

Listing 4: PD control loop in `controller_node.py`

```

1 def control_loop(self):
2     """PD control law executed at 100 Hz."""
3     now = self.get_clock().now()
4
5     if self.prev_time is None:
6         self.prev_time = now
7         return
8
9     dt = (now - self.prev_time).nanoseconds * 1e-9
10    if dt <= 0.0:
11        return
12    self.prev_time = now
13
14    # PD law
15    error = self.ref_position - self.cur_position
16    error_dot = (error - self.prev_error) / dt
17    self.prev_error = error
18
19    effort = KP * error + KD * error_dot
20    effort = max(-EFFORT_LIMIT, min(EFFORT_LIMIT, effort))
21
22    msg = Float64()
23    msg.data = effort
24    self.cmd_pub.publish(msg)

```

2.4 Reference Position Service (Part 2c)

A custom service `scara_ik/srv/SetJointPosition` was added to the existing `scara_ik` package (alongside `ScaraIK.srv`):

Listing 5: `SetJointPosition.srv`

```

1 float64 position
2 ---
3 bool success
4 string message

```

The service callback validates the requested position against joint limits before updating the controller setpoint:

Listing 6: Service callback in `controller_node.py`

```

1 def set_position_cb(self, request, response):
2     target = request.position
3
4     if not (J3_MIN <= target <= J3_MAX):
5         response.success = False
6         response.message = (
7             f'Requested {target:.4f} m outside limits '
8             f'[{J3_MIN}, {J3_MAX}] m.'
9         )
10        self.get_logger().warn(response.message)
11        return response
12
13    self.ref_position = target
14    response.success = True
15    response.message = f'Reference position set to {target:.4f} m'
16    self.get_logger().info(response.message)
17    return response

```

The service is called from the terminal as:

```

ros2 service call /scara/set_joint3_position \
  scara_ik/srv/SetJointPosition "{position: -0.1}"

```

and

```

ros2 service call /scara/set_joint3_position \
  scara_ik/srv/SetJointPosition "{position: 0.2}"

```

2.5 Plotting Current and Reference Postions of Joint 3 (Part 2d)

The reference and measured positions of joint_3 were recorded in a CSV file and then plotted in MATLAB against elapsed time.

Listing 7: CSV logging setup in controller_node.py

```

1 log_dir = os.path.expanduser('~/.ros2_ws/src/scara_controller/logs')
2 os.makedirs(log_dir, exist_ok=True)
3 log_path = os.path.join(log_dir, 'joint3_log.csv')
4 self.log_file = open(log_path, 'w', newline='')
5 self.log_writer = csv.writer(self.log_file)
6 self.log_writer.writerow(['time_s', 'ref_position', 'cur_position'])
7 self.start_time = self.get_clock().now()

```

For every control cycle, the elapsed time, reference position, and measured position are recorded in separate rows.

Listing 8: Logging a control cycle in control_loop

```

1 t_elapsed = (now - self.start_time).nanoseconds * 1e-9
2 self.log_writer.writerow([t_elapsed, self.ref_position, self.
   cur_position])
3 self.log_file.flush()

```

A clean shutdown is ensured with an additional `destroy_node`:

Listing 9: Clean shutdown

```

1 def destroy_node(self):
2     self.log_file.close()
3     super().destroy_node()

```

The MATLAB script reads the CSV file, extracts the time, reference position, and measured position data from it, then plots the reference position and measured position against time.

Listing 10: plot_joint3.m

```

1 log_dir = fullfile(getenv('HOME'), 'ros2_ws', 'src', '
   scara_controller', 'logs');
2 data = readmatrix(fullfile(log_dir, 'joint3_log.csv'));
3
4 t = data(:,1);
5 ref = data(:,2);
6 cur = data(:,3);
7
8 figure;
9 plot(t, ref, 'r--', 'LineWidth', 1.5); hold on;
10 plot(t, cur, 'b-', 'LineWidth', 1.5);
11 grid on;
12 xlabel('Time (s)');
13 ylabel('Joint 3 Position (m)');
14 title('Joint 3 Position Tracking: Reference vs. Measured');
15 legend('Reference', 'Measured', 'Location', 'best');

```

The resulting figure is shown in Figure 2.

2.6 Results

```
tracy@motherboard:~/rbe500$ Open chat Ctrl+I
Show suggestions Ctrl+Space
Don't show this again

---
array('d', [-0.0023120670337364903])
---
array('d', [-0.0028120670338181927])
---
array('d', [-0.0033120670338998904])
---
array('d', [-0.003812067033981583])
---
array('d', [-0.004312067034063271])
---
array('d', [-0.004812067034144953])
---
array('d', [-0.00531206703422663])
---
array('d', [-0.005812067034308303])
---
array('d', [-0.006312067034389971])
---
array('d', [-0.006812067034471634])
---
array('d', [-0.006877771877743165])
---
array('d', [-0.00651061476910804])
---
```

(a) Start: $d_3 = 0.0$ m

```
tracy@motherboard:~/rbe500$ ros2 service call /scara/set_joint3_
position scara_ik/srv/SetJointPosition "{position: -0.1}"
requester: making request: scara_ik.srv.SetJointPosition_Request
(position=-0.1)
response:
scara_ik.srv.SetJointPosition_Response(success=True, message='Re
ference position set to -0.1000 m')

array('d', [-0.10206928135563664])
---
array('d', [-0.10156928135544893])
---
array('d', [-0.10106928135526122])
---
array('d', [-0.1009536661113407])
---
array('d', [-0.1012210527779293])
---
```

(b) Reference: $d_3 = -0.1$ m

```
tracy@motherboard:~/rbe500$ ros2 service call /scara/set_joint3_
position scara_ik/srv/SetJointPosition "{position: 0.2}"
waiting for service to become available...
requester: making request: scara_ik.srv.SetJointPosition_Request
(position=0.2)
response:
scara_ik.srv.SetJointPosition_Response(success=True, message='Re
ference position set to 0.2000 m')

array('d', [0.1998726152811988])
---
array('d', [0.20000000000031684])
---
array('d', [0.2000000000007298])
---
array('d', [0.1999457101399574])
---
```

(c) Reference: $d_3 = 0.2$ m

Figure 1: Joint 3 position tracking three reference commands. The PD controller drives the prismatic joint to each target with no steady-state error.

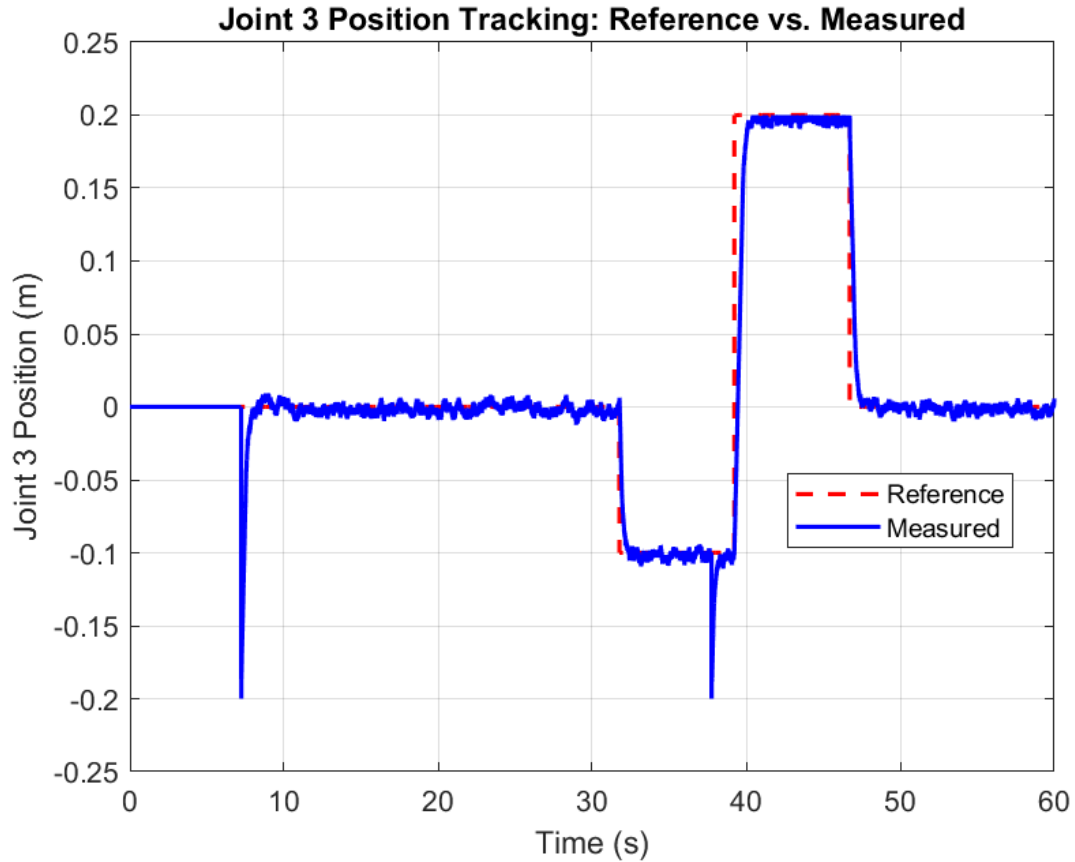


Figure 2: MATLAB plot of reference position vs. measured joint_3 position over time.

3 Package Structure

The submission adds two packages to the existing `ros2_ws/src/` workspace from PA1:

- `scara_description` — updated URDF (joints fixed, frame corrected, Gazebo plugins added) and updated launch file (effort bridge, controller node).
- `scara_controller` — PD controller node (`controller_node.py`, `ament_python`).
- `scara_ik` — extended with `SetJointPosition.srv`.